

"Ss. Cyril and Methodius" University in Skopje
FACULTY OF COMPUTER
SCIENCE AND ENGINEERING

JAVA Exceptions

Advanced Programming

FCSE 2026

Exceptions

- Due to errors in design or coding, our programs can fail in unexpected ways during execution. An **exception** is a condition which is caused by an error during the execution of the program. The purpose of the exception handling mechanism is to provide a means for detecting and reporting "exceptional circumstances".

Exception in Java

- In Java, an exception is an event that occurs during program execution and disrupts the normal flow of instructions. Instead of allowing a runtime error to crash the application, Java's exception mechanism provides a structured and robust way to handle these "exceptional circumstances.“
- The primary goal of Java's exception handling mechanism is to **separate** the **main application logic from the error-handling code**.

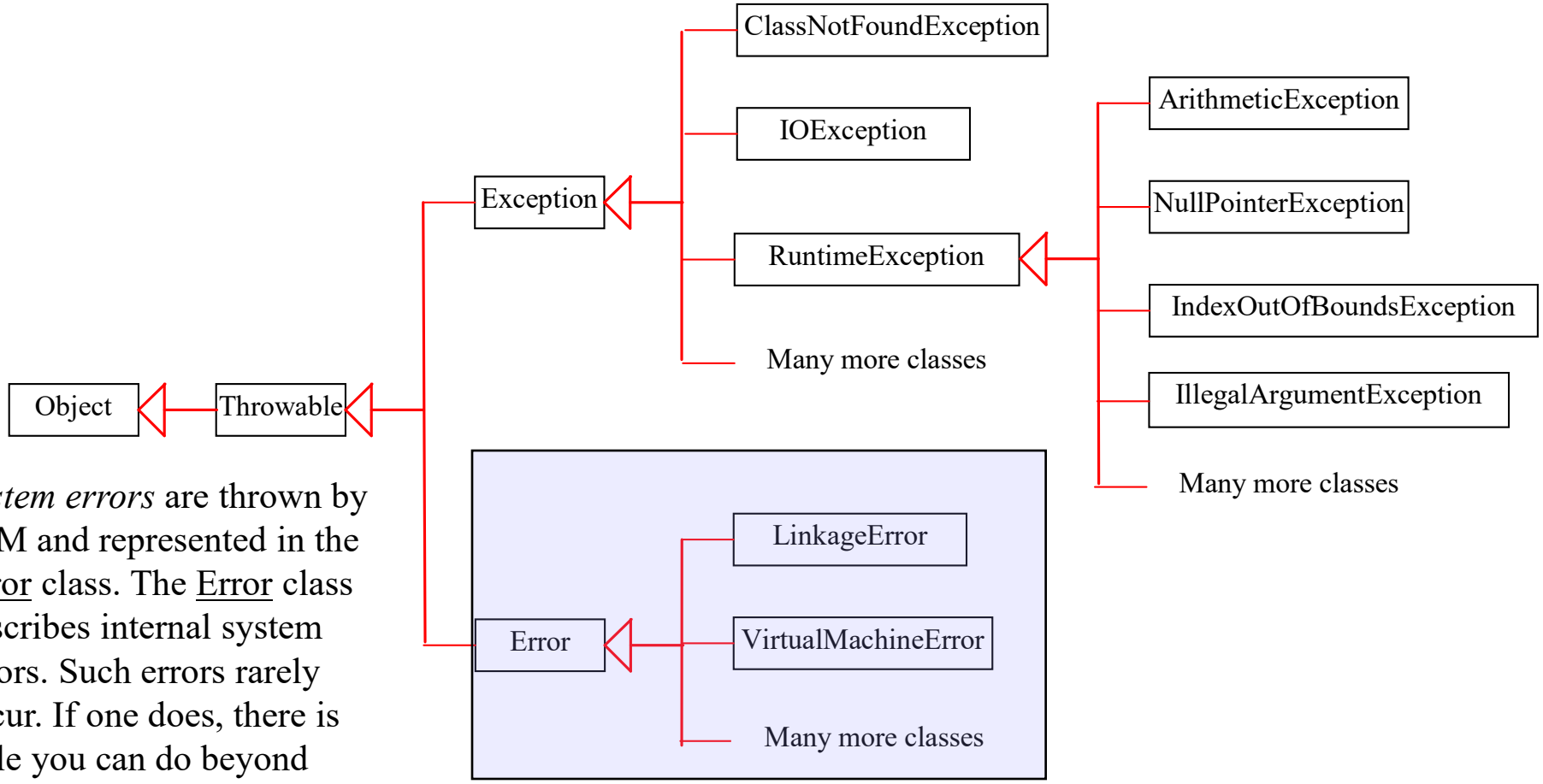
Exception causes

- Exceptions can be caused by a wide range of issues, including:
  - **Programmatic Errors**: Mistakes in code logic, such as trying to access an element outside the bounds of an array (ArrayIndexOutOfBoundsException) or using an object reference that is null (NullPointerException).
  - **User Errors**: Invalid input from a user, like entering text where a number is expected (NumberFormatException).
  - **Resource Failures**: External issues like a file not being found (FileNotFoundException), a network connection being lost, or the JVM running out of memory (OutOfMemoryError).

Exceptions

- An exception is an object that describes an **unusual or error situation**
- Exceptions are **thrown** out by the program, and can be **captured and managed** by **another part of the program**
- A program can be separated into normal runtime and runtime with exceptions
  - *Errors are also presented as objects in Java, but they are usually an irreparable situation that we should not deal with*

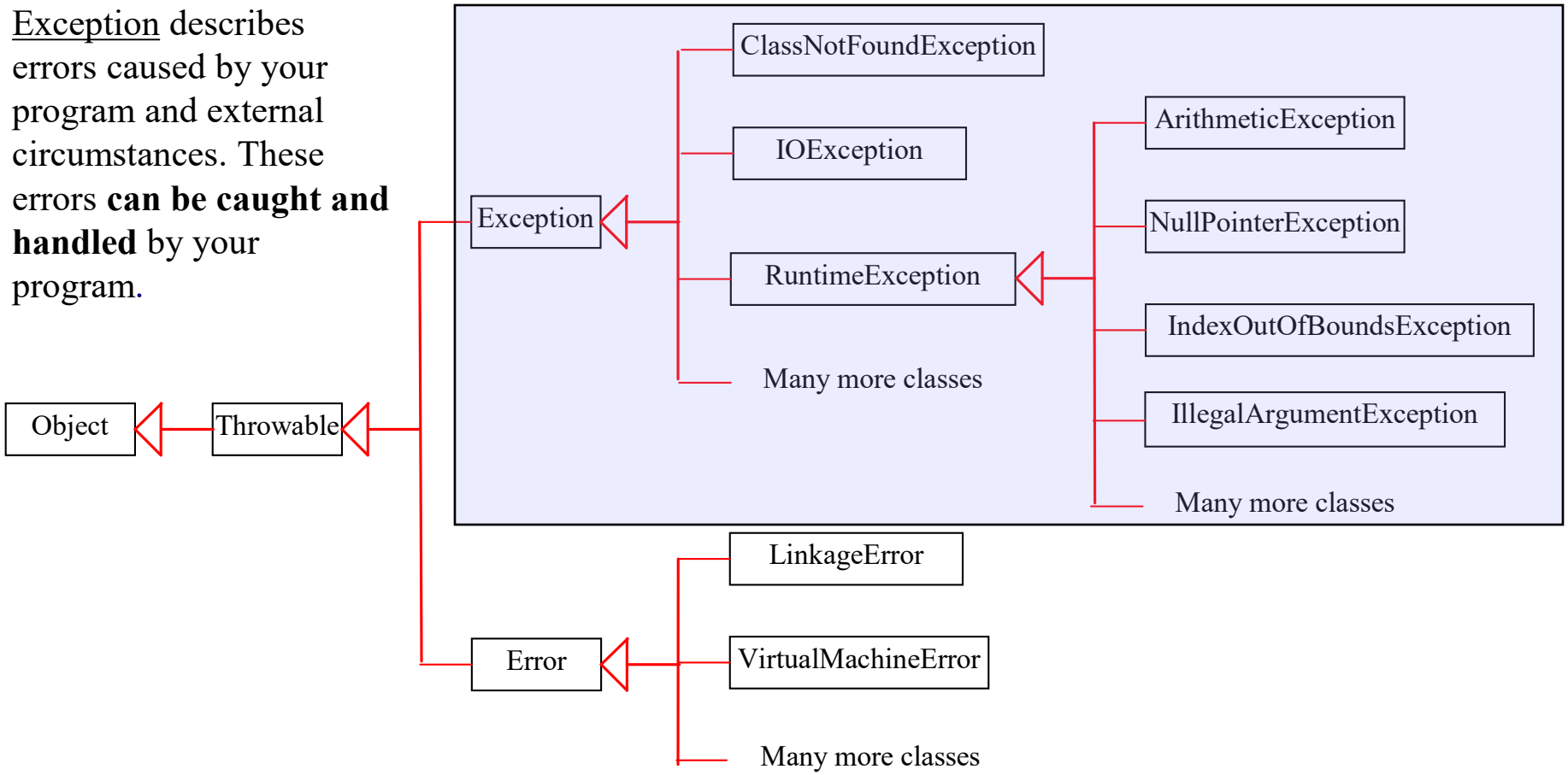
System Errors



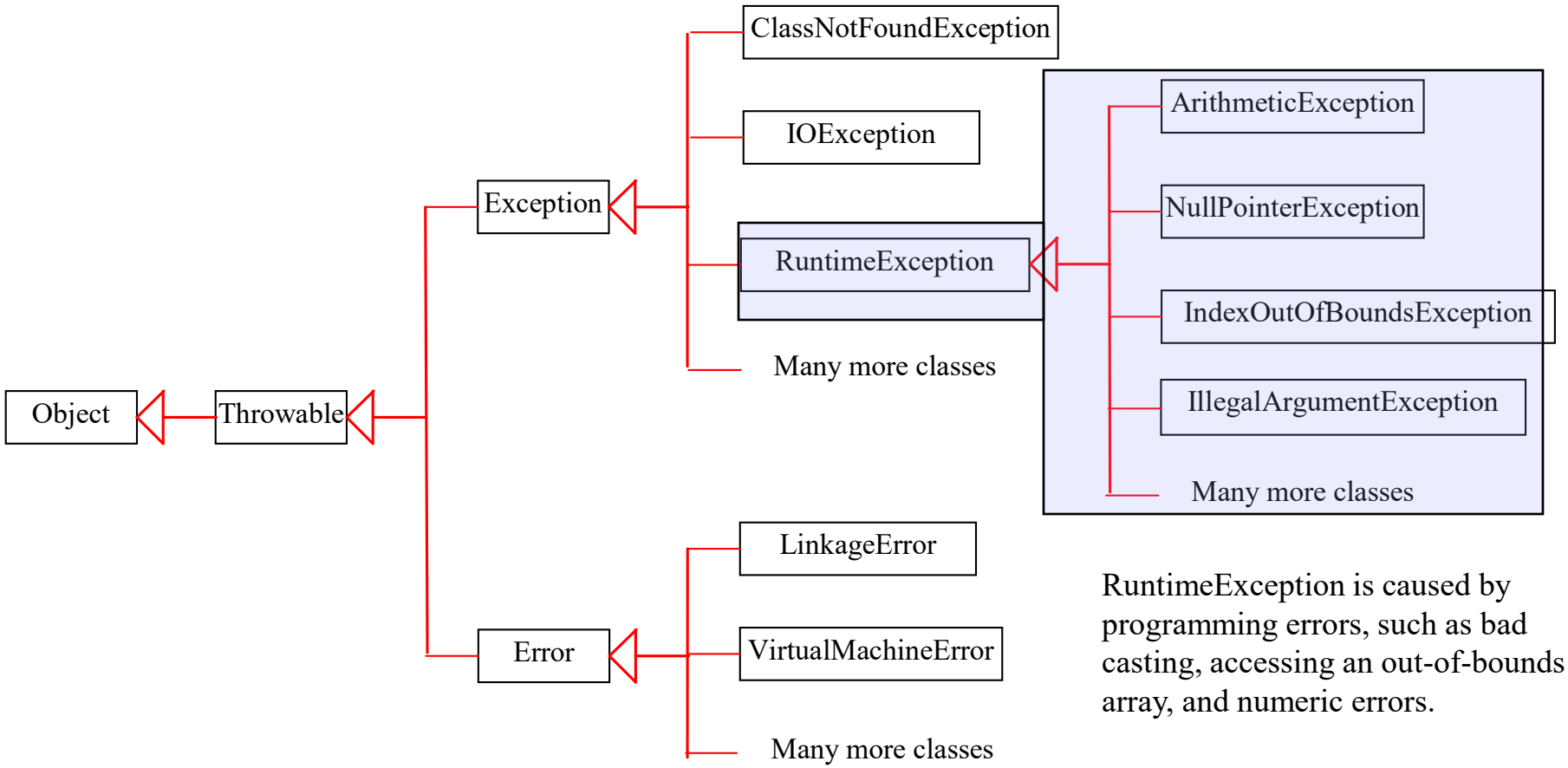
System errors are thrown by JVM and represented in the Error class. The Error class describes internal system errors. Such errors rarely occur. If one does, there is little you can do beyond **notifying the user** and trying to **terminate the program** gracefully.

Exceptions

Exception describes errors caused by your program and external circumstances. These errors **can be caught and handled** by your program.



Runtime Exceptions



RuntimeException is caused by programming errors, such as bad casting, accessing an out-of-bounds array, and numeric errors.

Don't catch it,

FIX IT!

Checked Exceptions vs. Unchecked Exceptions

RuntimeException, Error, and their subclasses are known as *unchecked exceptions*. All other exceptions are known as *checked exceptions*, meaning that the compiler **forces** the programmer **to check and handle exceptions**.

Unchecked Exceptions

In most cases, unchecked exceptions reflect errors in the programming logic that **cannot be corrected** (**at runtime** of the program). For example, a NullPointerException is thrown if you access an object through a reference variable before the object is assigned to it; IndexOutOfBoundsException is thrown if you access an element in an array outside the boundaries of the array. These are **logical errors** that need **to be corrected in the program**. *Unverifiable exceptions* may appear anywhere in the program. To avoid overuse of try-catch blocks, Java **does not require** you to write code **to catch uncheckable exceptions**.

try-throw-catch Mechanism

This mechanism is built around five keywords:

1. **try**: A block of code that **might throw an exception** is placed here.
2. **catch**: This block **"catches" and handles** a specific type of exception if one occurs within the try block.
3. **finally**: This block contains code that **will always execute**, whether an exception was thrown or not. It is primarily used for cleanup tasks, like closing files or network connections.
4. **throw**: Manually **throws an exception** to signal that an error condition has been met.
5. **throws**: Declares in a method's **signature that the method might throw** a particular type of exception.

try

- The **try** block contains the code of the underlying algorithm
- It is called a **try** block because it “tries" to execute the planned code
- It may contain the code that generates the exception if something unusual happens

```
try
{
   CodeThatMayThrowAnException
}
```

throw

```
throw new
   ExceptionClassName(PossiblySomeArguments);
```

- When an exception is thrown, **the** code **execution** in the try block **stops**
  - Typically, the flow of control is transferred to another piece of code called *the catch block*
- The value generated as an argument through the throw operator is always an **object of some Exception class**
  - The execution of the **throw statement** is called *throwing an exception*

`catch`

- When the exception is thrown, the catch block begins to execute
  - The catch block has one parameter
  - An object that is generated as an exception is carried as a parameter of the catch block
- Executing a catch block is called *catching an exception* or dealing with an exception
  - When an exception is "thrown", it should be "caught" in the catch block

catch

```
catch(Exception e)
{
   ExceptionHandlingCode
}
```

- The catch block looks like a method definition that has one parameter of the Exception class
  - It's not a true definition of a method, but
- A catch block is a separate piece of code that is executed when a program executes a throw command in the corresponding try block
  - A catch block is usually referred to as an *exception-dealing block*
  - It can have **at most one parameter**

`try`

- **Two outcomes are possible** while executing the try block:

1. **No exception** will be **thrown** into the try block
   - The code in the try block will be executed completely (until the end of the block)
   - The catch block will not be executed
   - Program execution **resumes** normally with **code after the catch block**

try

2. An **exception is thrown** into the try block and caught in the catch block
   - The rest of **the code** in the try block **after the exception** has been thrown **is not executed**
   - The **control is transferred** to the appropriate catch block
   - The **object thrown** as an exception is passed as the **parameter** of the catch block
   - The code from the **catch block is executed**
   - The code that follows the catch block is executed (if any)

Exception classes of standard packages

- A number of predefined exception classes are found in some Java standard package
  - Example:

```
IOException
NoSuchMethodException
FileNotFoundException
```

- Certain exception classes must be included in order to be used

```
import java.io.IOException;
```

Standard packages exceptions classes

- The predefined exception class **`Exception`** is the parent class of all exception classes
  - Each exception class is derived from the **`Exception`** class
  - An exception class can be used directly in a class or used to define a derived class
  - The **`Exception`** class is in the **`java.lang`** package and does not require an import command

`getMessage` method

```
. . . // method code
try
{
   . . .
   throw new Exception(StringArgument);
   . . .
}
catch(Exception e)
{
   String message = e.getMessage();
   System.out.println(message);
   System.exit(0);
}   . . .
```

Defining Exception Classes

- The `throw` command can throw an exception (object) that can be instance of any exception class
- In addition to predefined Java exceptions, **custom exception classes** can be created by the programmer to **provide more meaningful or specific error information** to the corresponding `catch` block.
- **Different exception types** should be defined to **represent different kinds of exceptional situations**, improving clarity and maintainability of the code.

Defining Custom Exception Classes

- Any **custom exception class** must be derived from an existing exception class — typically **`Exception`** or another **`Exception`**-derived class.
- A custom exception can **extend** any standard Java exception class or another **user-defined exception**.
- The **constructors** are the most important elements to define in a custom exception class.
- They should properly **initialize** or **pass parameters** to the constructors and members **inherited from the base class**.
- Typically, **no additional fields or methods** are required beyond those inherited — unless the exception needs to **store extra context information**.

Exception class

Display 9.3 A Programmer-Defined Exception Class

```
1  public class DivisionByZeroException extends Exception
2  {
3      public DivisionByZeroException()
4      {
5          super("Division by Zero!");
6      }
7      public DivisionByZeroException(String message)
8      {
9          super(message);
10     }
11 }
```

You can do more in an exception constructor, but this form is common.

super is an invocation of the constructor for the base class Exception.

Custom Exception Classes

- An exception class constructor can be defined to accept an argument of a different type (other than the predefined string type)

Exception class with int message

Display 9.5 An Exception Class with an int Message

```
1  public class BadNumberException extends Exception
2  {
3      private int badNumber;

4      public BadNumberException(int number)
5      {
6          super("BadNumberException");
7          badNumber = number;
8      }

9      public BadNumberException()
10     {
11         super("BadNumberException");
12     }

13     public BadNumberException(String message)
14     {
15         super(message);
16     }

17     public int getBadNumber()
18     {
19         return badNumber;
20     }
21 }
```

Defining Exception Classes

- **Exception classes** can be **defined by the programmer**, but each custom exception class must **inherit from** an existing exception class.
- The `Exception` class is typically used as the base class, unless another existing exception type is **more appropriate**.
- At least **two constructors** should be defined — **sometimes more**, depending on the exception's purpose.
- When defining a custom exception, keep in mind that the `getMessage()` method is inherited from the base class and can be used to retrieve the exception's description.

The `getMessage()` Method and Exception Constructors

- For all **predefined Java exception classes**, the `getMessage()` method returns the **string** passed as an argument to the constructor when the exception is created.
- If a **no-argument constructor** is used, it returns a **default predefined message**.
- This behavior should be **preserved** in all **user-defined exception classes**.
- Therefore, each custom exception class should include:
  - A **constructor with one `String` parameter**, whose body begins with a call to `super(message)`, passing the received argument.
  - A **no-argument constructor**, whose body begins with a call to `super("default message")` to provide a predefined message.

Multiple `catch` blocks

- Each `catch` block can handle exceptions **of the type specified in its header**.
- Multiple exception types can be caught by using multiple catch blocks following a single `try` block.
- There is no limit to the number of `catch` blocks, but **their order is important** —more specific exceptions should be caught before more general ones.
- When an exception is thrown from the `try` block, the `catch` blocks **are evaluated in order**
- The **first one that matches** the type of exception that is thrown **will be executed**

Order of exception types in catch

```
catch (Exception e)
{ . . . }
catch (NegativeNumberException e)
{ . . . }
```

- Because **NegativeNumberException** is a derived class from **Exception**, all **NegativeNumberExceptions** will be caught by the first catch block before reaching the second block
- The catch block for **NegativeNumberException** will never be executed!
- For the correct order, simply change the order of the two catch blocks

Declaring Exceptions with the `throws` Clause

- If a method **throws an exception** that **it does not handle,** it must **declare** that it may throw that exception type.
- This declaration is made using a **`throws` clause,** which acts as a **warning to the caller** of the method.
- The process of listing exception classes in the **`throws`** clause is called **exception declaration.**
- The following example shows that the method **`aMethod()`** may throw an exception of type **`AnException`**:

```
public void aMethod() throws AnException
```

- If a method can throw **multiple exceptions,** they should be **separated by commas**

```
public void aMethod() throws AnException, AnotherException
```

- When a method **throws an exception** and **does not handle it,** the **execution of that method immediately terminates** at the point where the exception is thrown.

Ways to Handle Exceptions in a Method

There are **two ways** to handle exceptions that may be thrown within a method:

1. **Handle the exception inside the method**
   - Place the code that might `throw` an exception inside a `try` block and handle the possible exception in a `catch` block within the same method.
2. **Declare the exception in the method header**
   - Specify the potential exceptions in a `throws` clause at the beginning of the method definition, indicating that the method may throw these exceptions to be handled by the caller.

Checked and Unchecked Exceptions

- Exceptions that must be **caught** or **declared** are called **checked exceptions**.
- The **compiler verifies** that these exceptions are either handled with a **`catch`** **block** or declared in the **`throws`** **clause**.
- The classes **`Throwable`**, **`Exception`**, and all classes derived from **`Exception`** (except those under **`RuntimeException`**) are checked exceptions.

Checked and Unchecked Exceptions

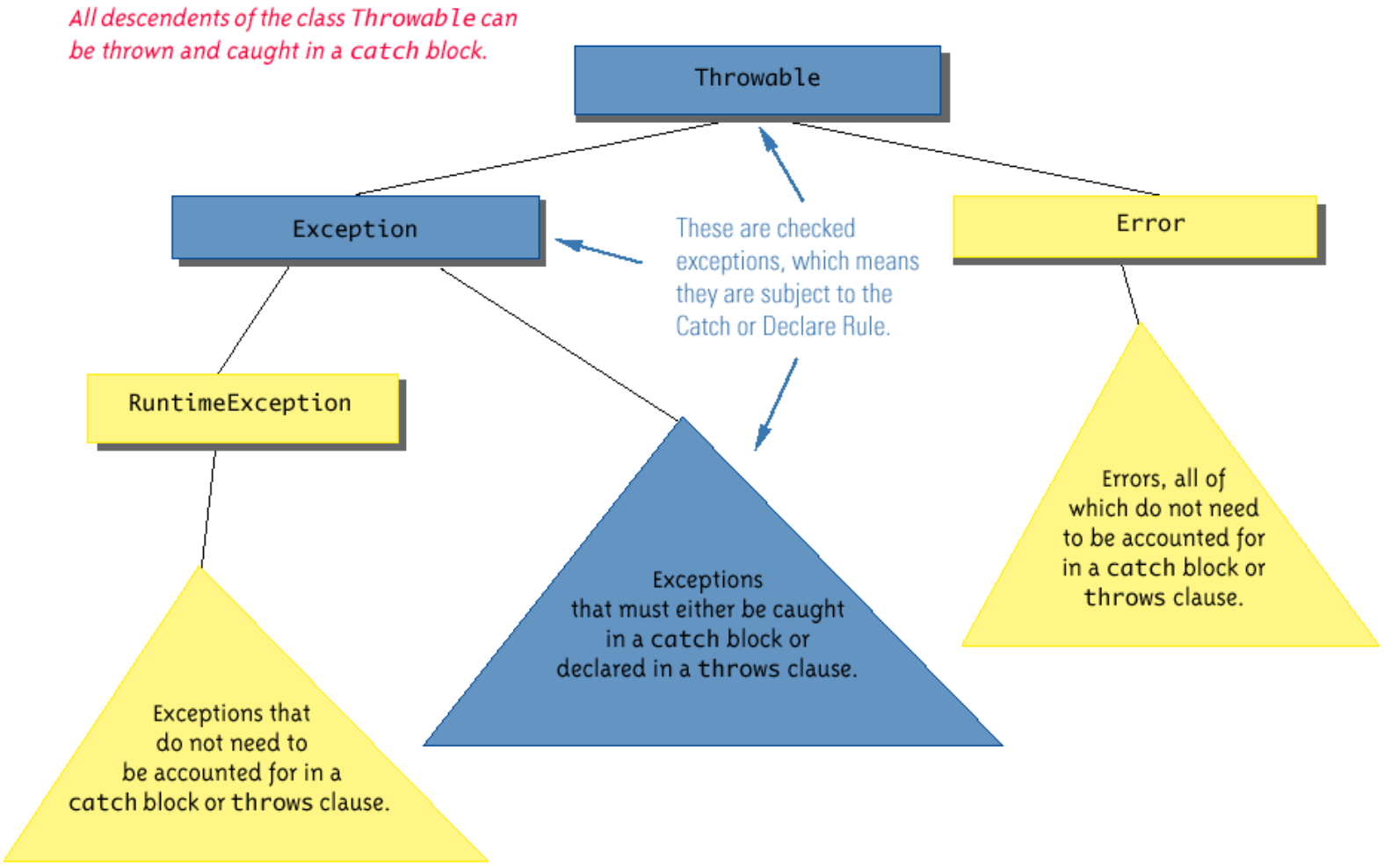
- All other exceptions are called **unchecked exceptions**.
- The class **`Error`** and all classes **derived from** it are known as error classes.
- **Error classes** are **not meant** to be caught or declared; they represent serious system failures.

Checked and Unchecked Exceptions

- **Checked exceptions** must follow the **"catch or declare" rule** —programs that might throw them will **not compile** until the exception is **properly handled or declared.**
- **Unchecked exceptions** (usually derived from `RuntimeException`) are exempt from this rule. They typically indicate **programming errors**, and the code must be **corrected** rather than explicitly handling them.

Hierarchy of Throwable Objects

Display 9.10 Hierarchy of Throwable Objects



The throws clause and the derived classes

- When a method is **overridden** in a derived (subclass) class, its **`throws` clause** must declare the **same exceptions** as the method in the base (superclass) —or a subset of those exceptions.
- The **overriding method cannot introduce new checked exceptions** that are not declared in the base class method.
- However, it **may omit** (remove) some of the exceptions declared in the base class method.
- This rule ensures that code using the base class interface can safely work with subclass objects **without handling unexpected exceptions**.

Using exceptions

- Example of a method that generates an exception:

```
public void someMethod()
                    throws SomeException
{
   . . .
   throw new
          SomeException(SomeArgument);
   . . .
}
```

Using exceptions

- When **someMethod** is used by **otherMethod**, the **otherMethod** must handle the exception:

```
public void otherMethod()
{
   try
   {
      someMethod();
      . . .
   }
   catch (SomeException e)
   {
      CodeToHandleException
   }
   . . .
}
```

Catch or Declare Checked Exceptions

- Java **requires** you to handle all **checked exceptions**.
- If a method declares that it can throw a checked exception, (i.e., an exception other than **`Error`** or **`RuntimeException`**), you must either:

1. **Handle it** using a try-catch block, or
2. **Declare** that your method also throws that exception using a **`throws`** **clause**.

- For example, suppose method `p1()` calls method `p2()`, and `p2()` may throw a checked exception such as **`IOException`**. In that case, the code must be written as shown in either (a) or (b):

```
void p1() {
  try {
    p2();
  }
  catch (IOException ex) {
    ...
  }
}
```

(a)

```
void p1() throws IOException {

  p2();

}
```

(b)

The `finally` block: Guaranteed Execution

- The **`finally`** block defines a section of code **that is guaranteed to be executed** after the **`try`** block, regardless of the outcome. Its primary purpose is to perform essential cleanup operations to prevent resource leaks.
  - **If used**, the **`finally`** block is placed after the **`catch`** blocks that follow the **`try`** block

```
try
{   .  .  .   }
catch(ExceptionClass1 e)
{   .  .  .   }
   .  .  .
catch(ExceptionClassN e)
{   .  .  .   }
finally
{
   CodeToBeExecutedInAllCases
}
```

Execution flow of a `try-catch-finally` block

When a method contains a `try-catch-finally` structure, the program flow is managed in one of three distinct ways, ensuring predictable behavior even when errors occur.

1. **Normal Execution (No Exception)**
2. **Handled Exception (Exception is Caught)**
3. **Unhandled Exception (Exception is Not Caught)**

Execution flow of a `try-catch-finally` block

Normal Execution (No Exception) – the "happy path" where everything succeeds

- The code inside the try block is executed from top to bottom. **No exception is thrown**, so all **catch blocks are skipped** and the finally block is executed.
- The method **continues execution** with the code immediately following the finally block.

Execution flow of a `try-catch-finally` block

Handled Exception (Exception is Caught) - An error occurs, but the program is prepared to handle it gracefully.

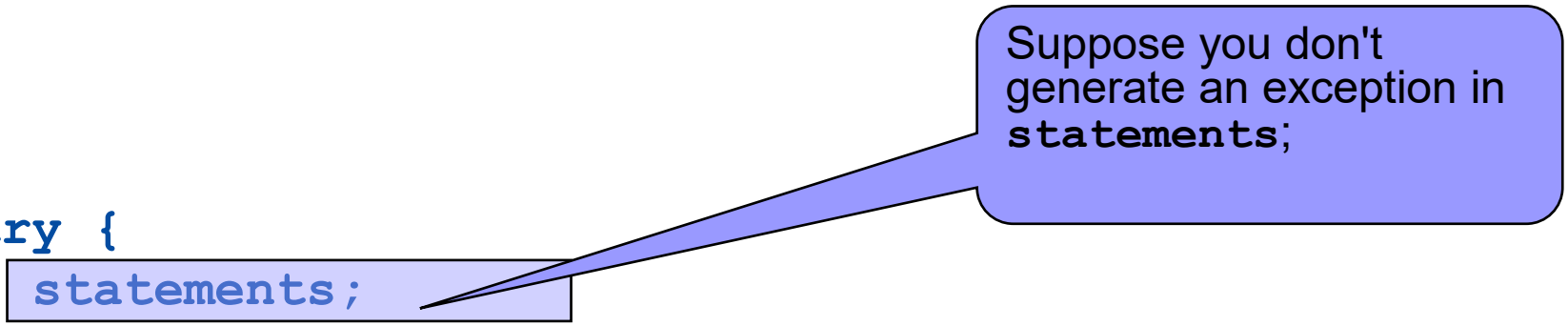
- An **exception is thrown**. The **execution of the try block stops immediately** at that point. The first matching block in the **catch clauses is executed**. The finally block is executed.
- The method **continues execution** with the code immediately following the finally block.

Execution flow of a `try-catch-finally` block

Unhandled Exception (Exception is Not Caught) - An unexpected error occurs with no specific handler in this method

- An **exception is thrown**. The execution of the try block **stops immediately**. The JVM searches for a **matching catch** block but **none is found**. The finally block is executed. This is the last chance for this method to perform cleanup.
- After the finally block completes, **the method abruptly terminates**, and the **exception is propagated** (passed up) to the method that called this one, to see if it can be handled there.

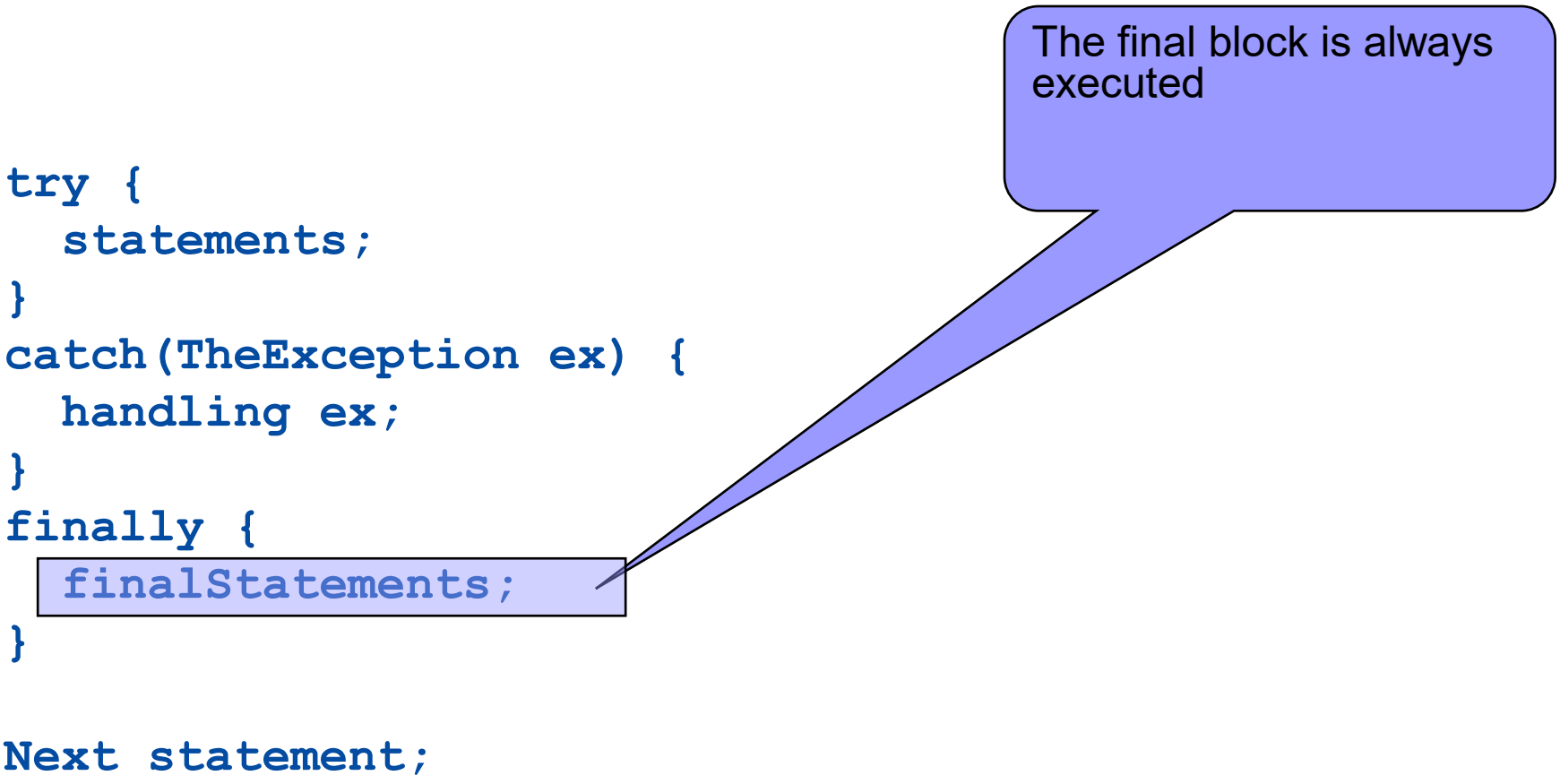
Tracing program execution (Normal Execution)




```
try {
  statements;
}
catch(TheException ex) {
  handling ex;
}
finally {
  finalStatements;
}

Next statement;
```

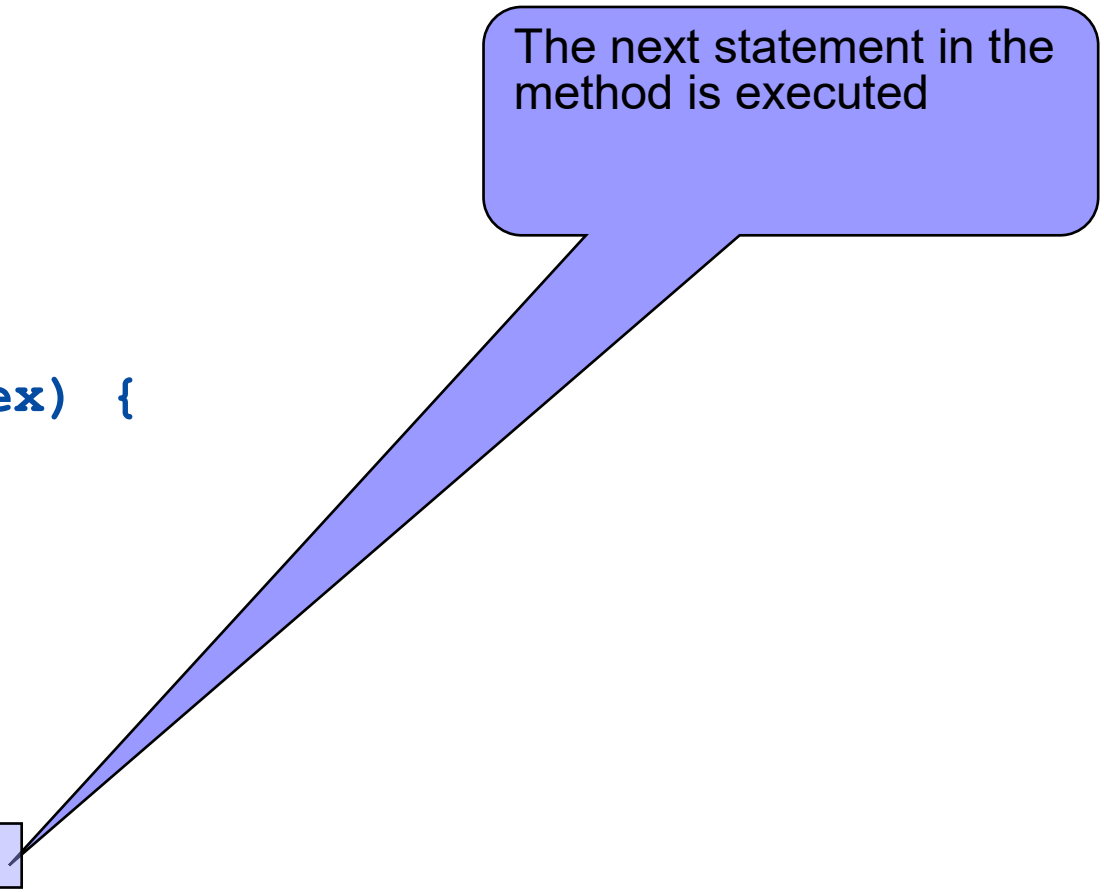
Tracing program execution (Normal Execution)



Tracing program execution (Normal Execution)

```
try {
  statements;
}
catch(TheException ex) {
  handling ex;
}
finally {
  finalStatements;
}

Next statement;
```



Tracing program execution (Handled Exception)

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
finally {
  finalStatements;
}

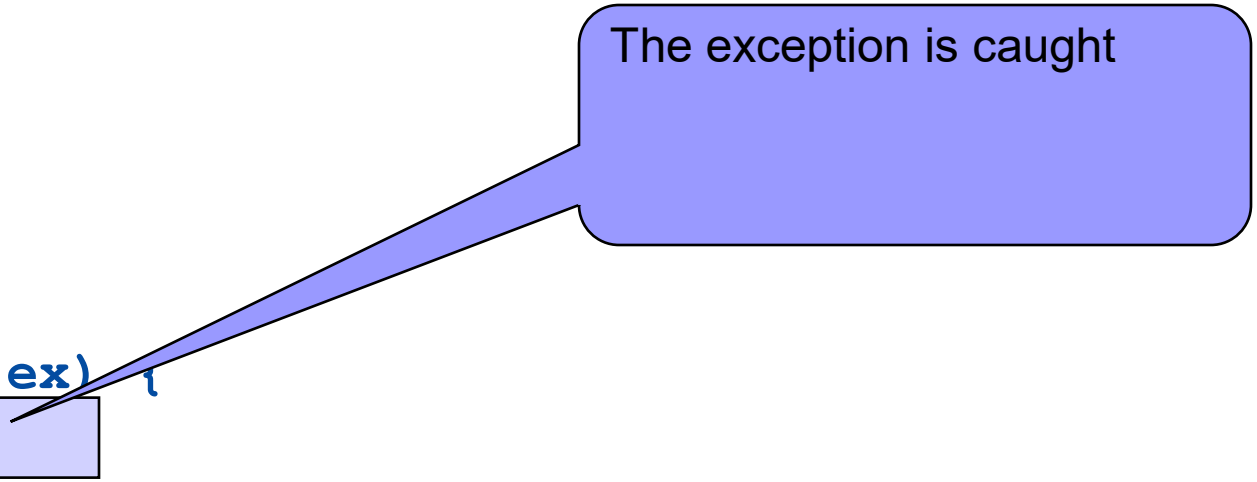
Next statement;
```

Suppose an exception of type **Exception1** is thrown into **statement2**

Tracing program execution (Handled Exception)

```
try {
   statement1;
   statement2;
   statement3;
}
catch(Exception1 ex) {
   handling ex;
}
finally {
   finalStatements;
}

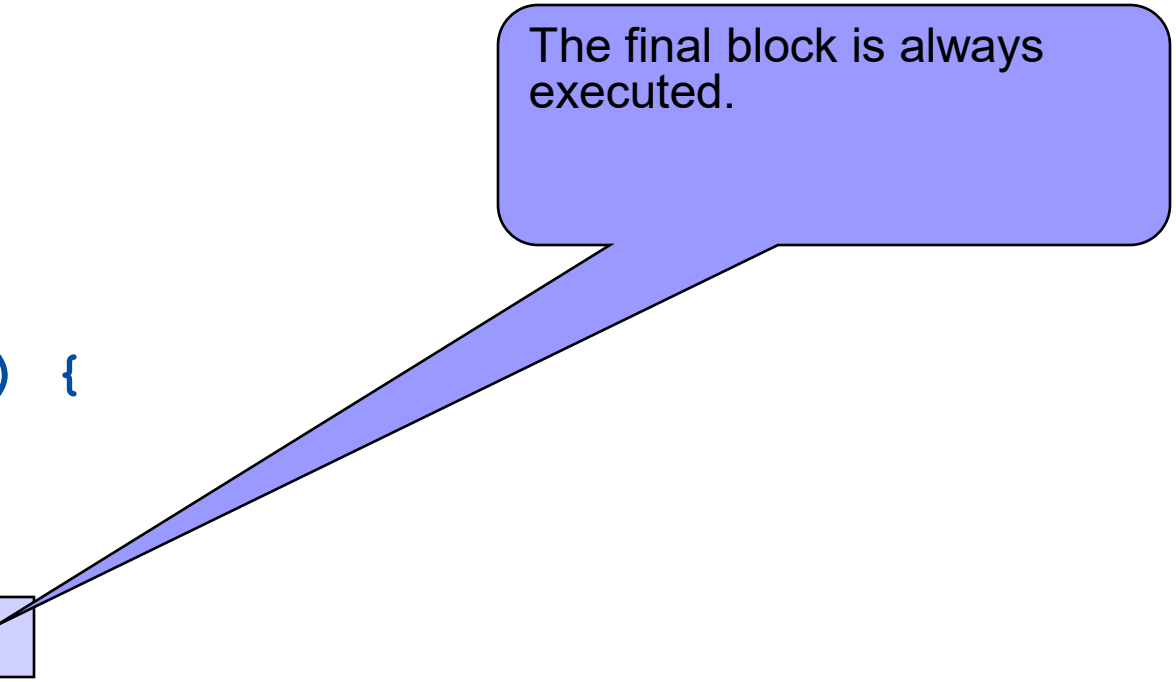
Next statement;
```



Tracing program execution (Handled Exception)

```
try {
   statement1;
   statement2;
   statement3;
}
catch(Exception1 ex) {
   handling ex;
}
finally {
   finalStatements;
}

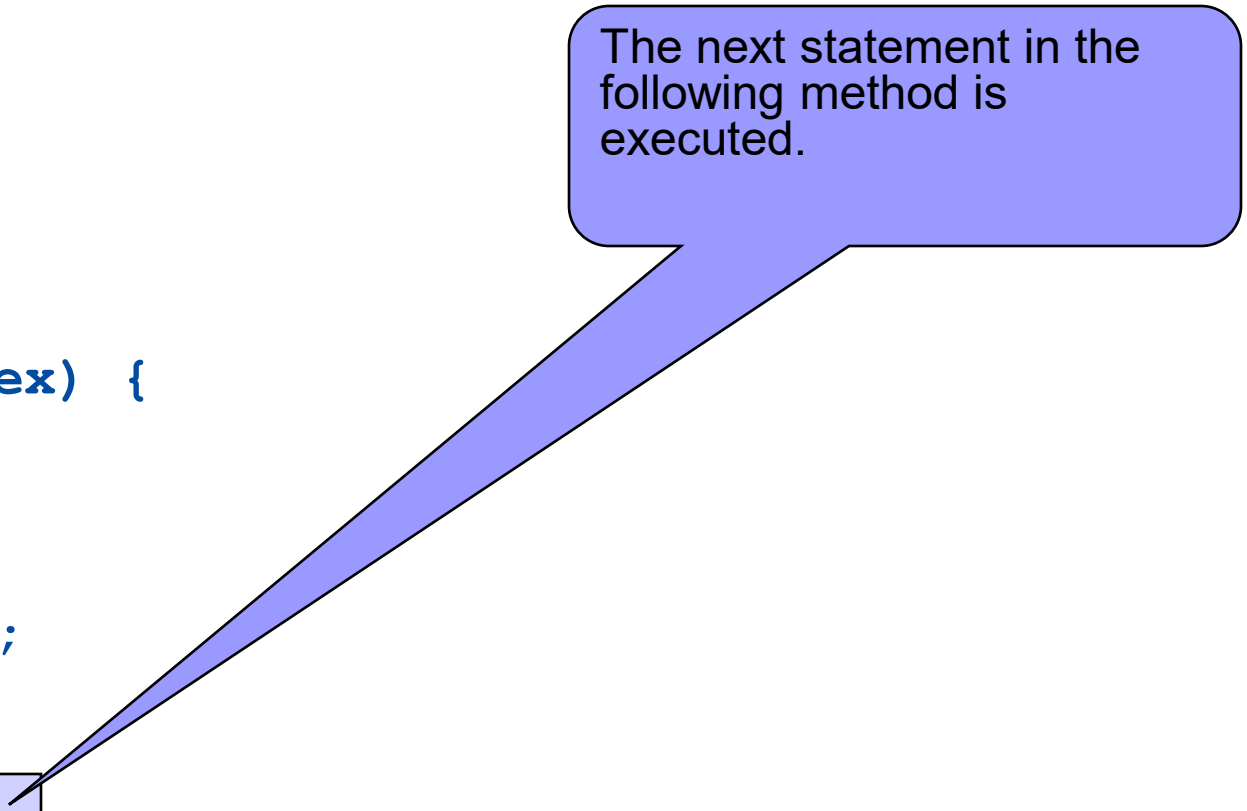
Next statement;
```



Tracing program execution (Handled Exception)

```
try {
   statement1;
   statement2;
   statement3;
}
catch(Exception1 ex) {
   handling ex;
}
finally {
   finalStatements;
}

Next statement;
```



Tracing program execution (rethrowing an exception)

```
try {
   statement1;
   statement2;
   statement3;
}
catch(Exception1 ex) {
   handling ex;
}
catch(Exception2 ex) {
   handling ex;
   throw ex;
}
finally {
   finalStatements;
}

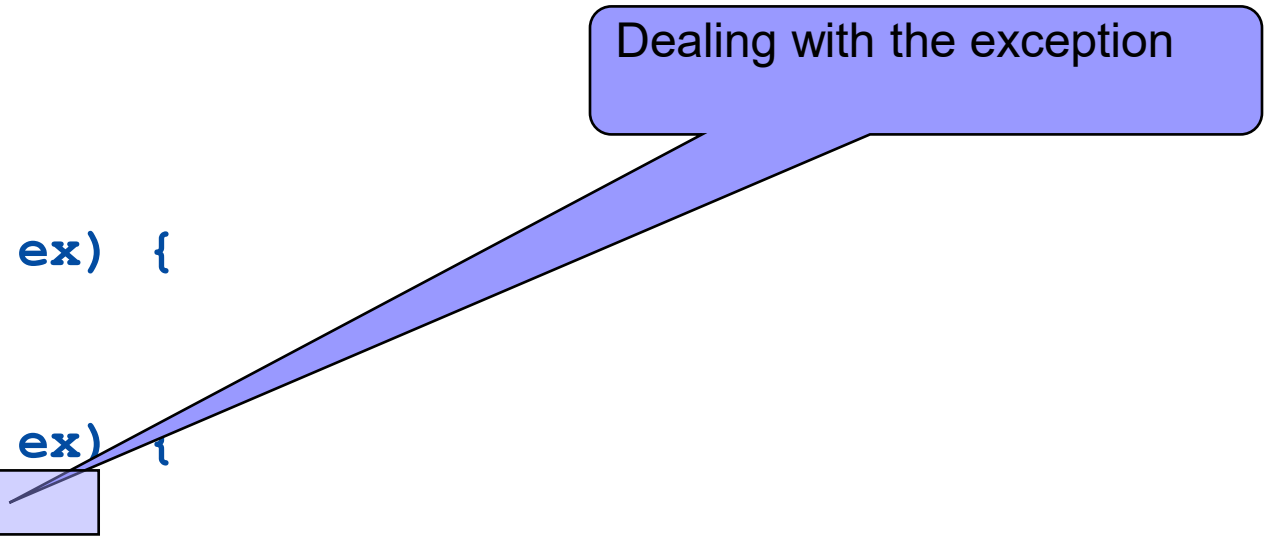
Next statement;
```

statement2 throws an exception of type **Exception2**.

Tracing program execution (rethrowing an exception)

```
try {
   statement1;
   statement2;
   statement3;
}
catch(Exception1 ex) {
   handling ex;
}
catch(Exception2 ex) {
   handling ex;
   throw ex;
}
finally {
   finalStatements;
}

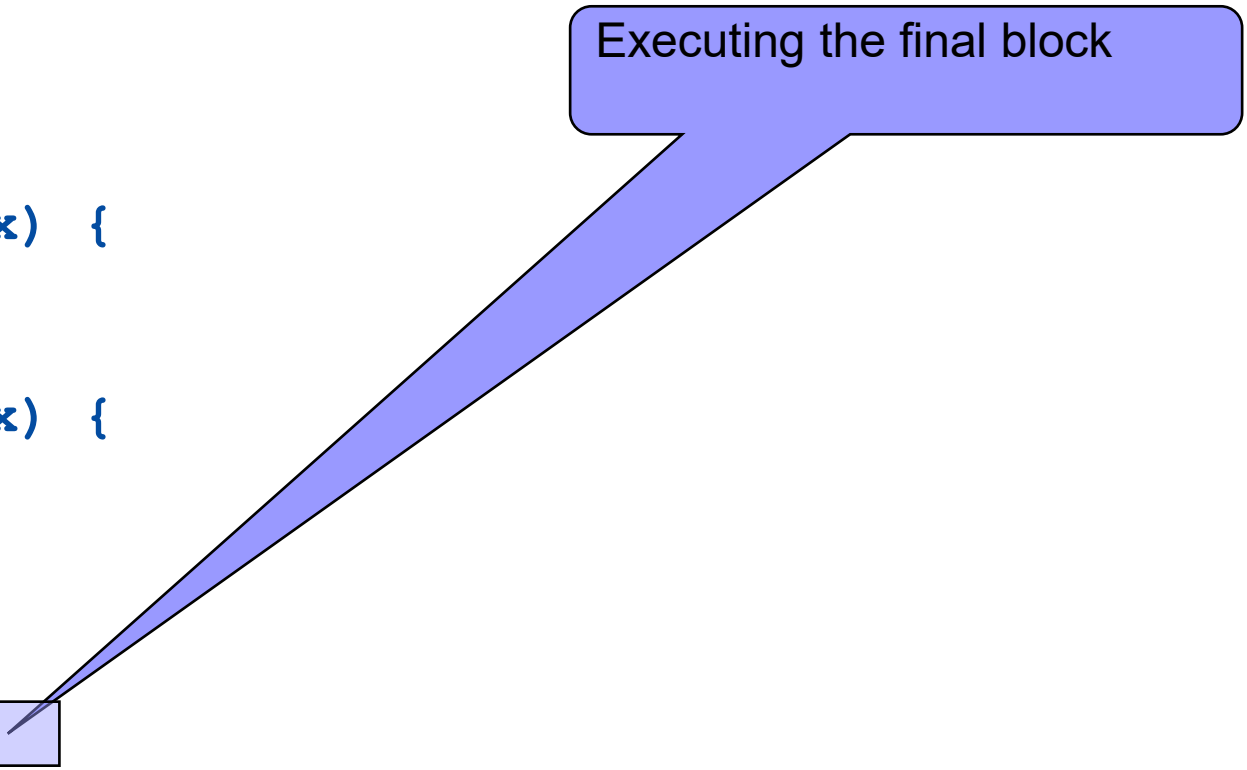
Next statement;
```



Tracing program execution (rethrowing an exception)

```
try {
   statement1;
   statement2;
   statement3;
}
catch(Exception1 ex) {
   handling ex;
}
catch(Exception2 ex) {
   handling ex;
   throw ex;
}
finally {
   finalStatements;
}

Next statement;
```



Tracing program execution

```
try {
   statement1;
   statement2;
   statement3;
}
catch(Exception1 ex) {
   handling ex;
}
catch(Exception2 ex) {
   handling ex;
   throw ex;
}
finally {
   finalStatements;
}

Next statement;
```

Rethrow the exception whereby control is passed to the caller method

Rethrowing exceptions

- A **`catch`** block can contain code that throws an exception
- Sometimes it is useful to catch the exception and then, depending on the message received by **`getMessage`** (or according to another criterion), throw the same exception or some other type of exception into the block sequence

ArrayIndexOutOfBoundsException

- An **ArrayIndexOutOfBoundsException** is a common runtime error in Java that occurs when a program attempts to access an array using an invalid index (outside the bounds of the array).
  - This normally leads to the termination of the program.
- Like all other derived classes from the RuntimeException class, it is an unchecked exception.
  - We are not required to handle it.
- When this exception is thrown, it indicates that there is an error in the program.
  - Instead of handling the exception, it is necessary to review and fix the error in the program.

Don't Catch It, Fix It!


Questions?